

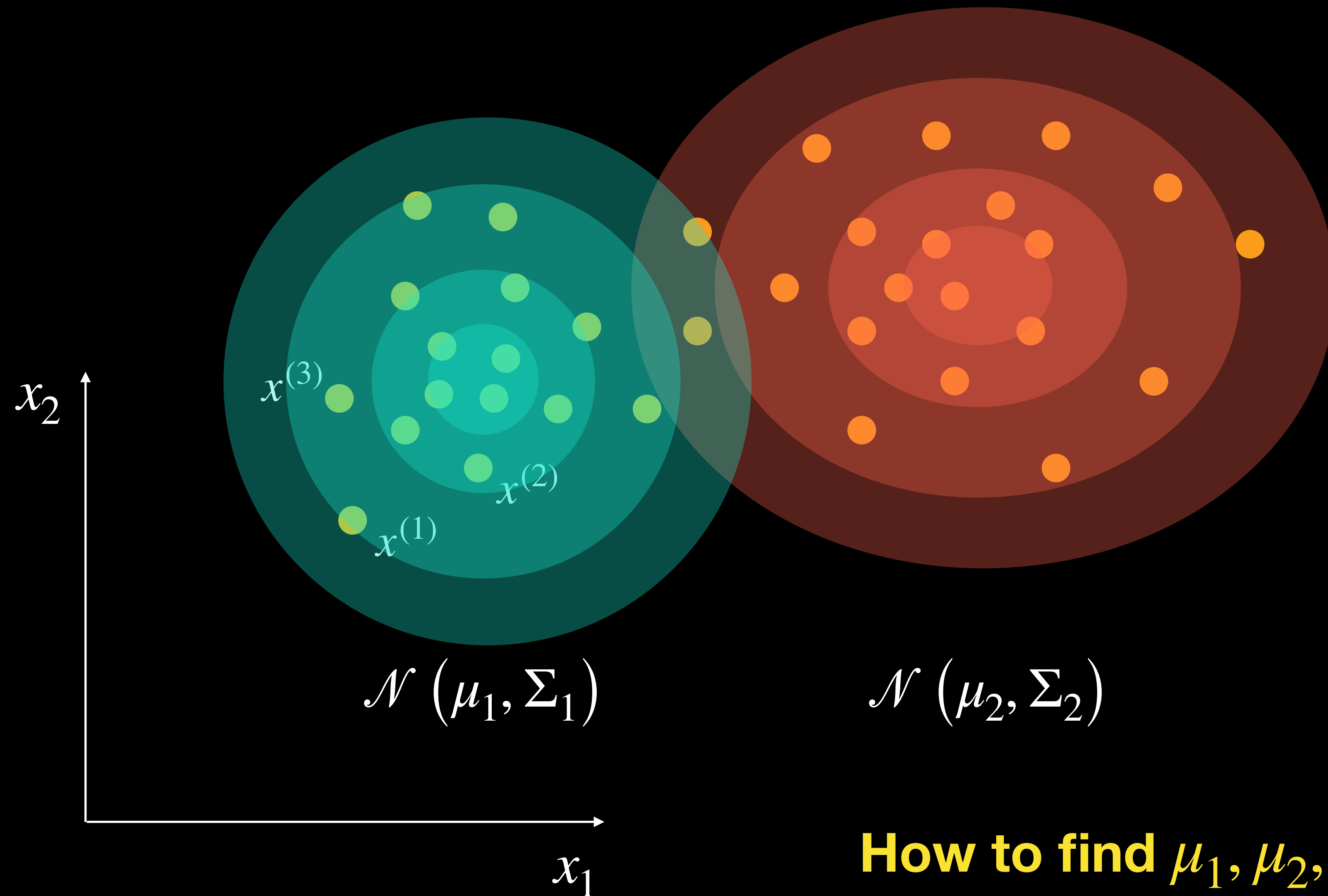
Gaussian Mixture Models

Prepared by: Joseph Bakarji

Modeling data as a Mixture of Gaussians

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...

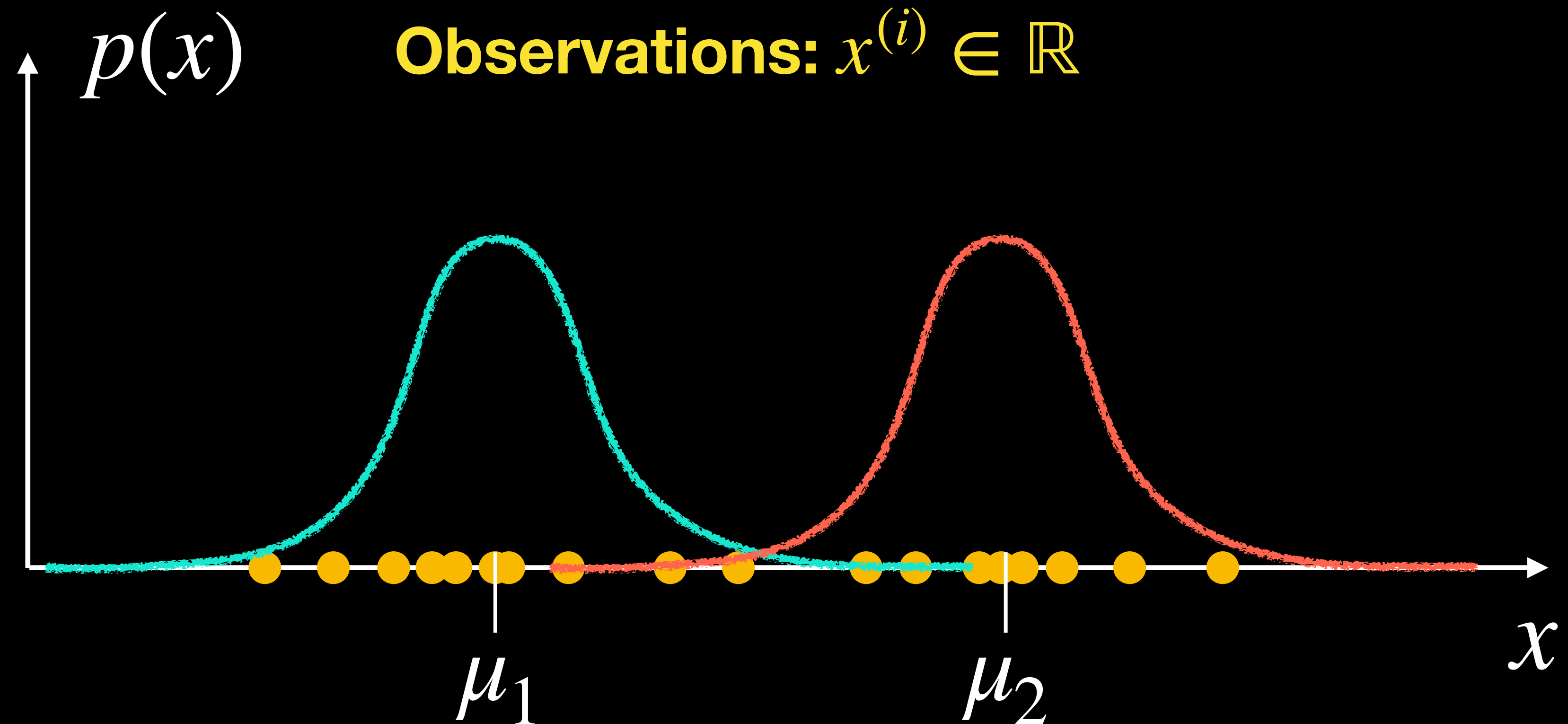


How to find $\mu_1, \mu_2, \Sigma_1, \Sigma_2$?

Modeling data as a Mixture of Gaussians

Dataset

x
1.2
3.2
4.3
3.2
...



Assume:

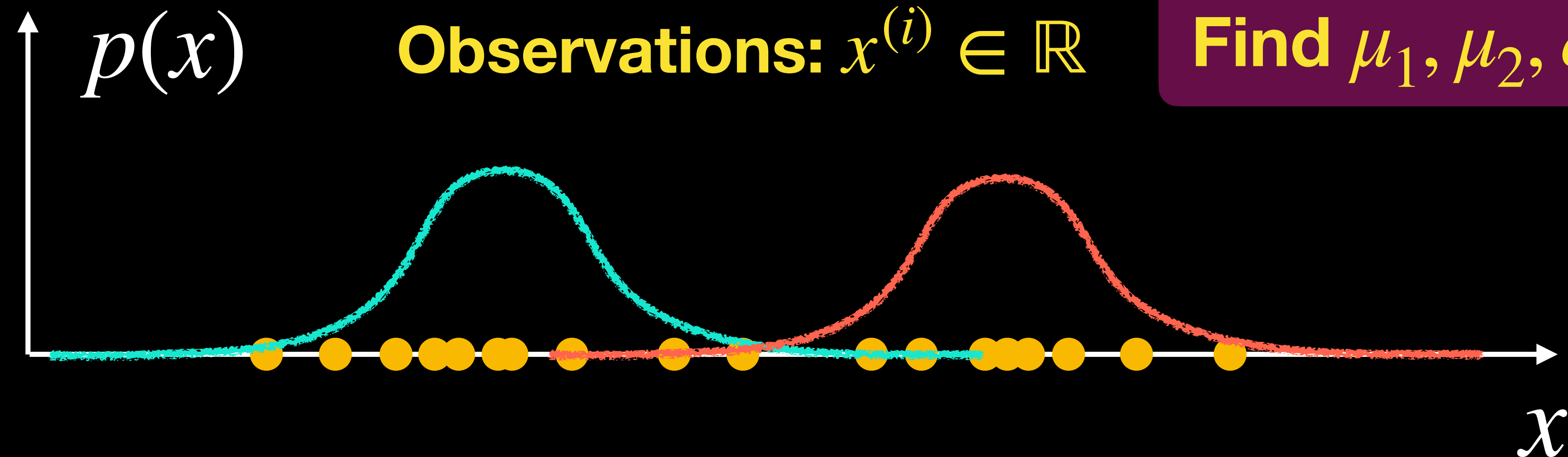
- Data is modeled by mixture of gaussians: $\mathcal{N}(\mu_i, \sigma_i^2)$
- We know how many gaussians we need

Find $\mu_1, \mu_2, \sigma_1, \sigma_2$

Modeling data as a Mixture of Gaussians

Dataset

x
1.2
3.2
4.3
3.2
...



Find $\mu_1, \mu_2, \sigma_1, \sigma_2$

Given: $x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(n)}$, and $k > 0$: number of clusters

Find

$$P(z^{(i)} = j)$$

Soft Assignment

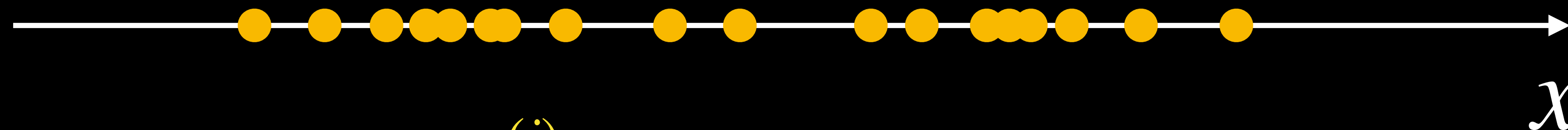
Probability that point i belongs to cluster j

$z^{(i)}$ is not directly observed: Latent variable

Gaussian Mixture Model

Dataset

x
1.2
3.2
4.3
3.2
...



Observations: $x^{(i)} \in \mathbb{R}$

Num. of Clusters: $k > 0$ clusters

Soft assignments: $z^{(i)}$

$$P(x^{(i)}, z^{(i)}) = P(x^{(i)} | z^{(i)}) P(z^{(i)})$$

$$P(z^{(i)}) = \text{Multinomial}(\phi)$$

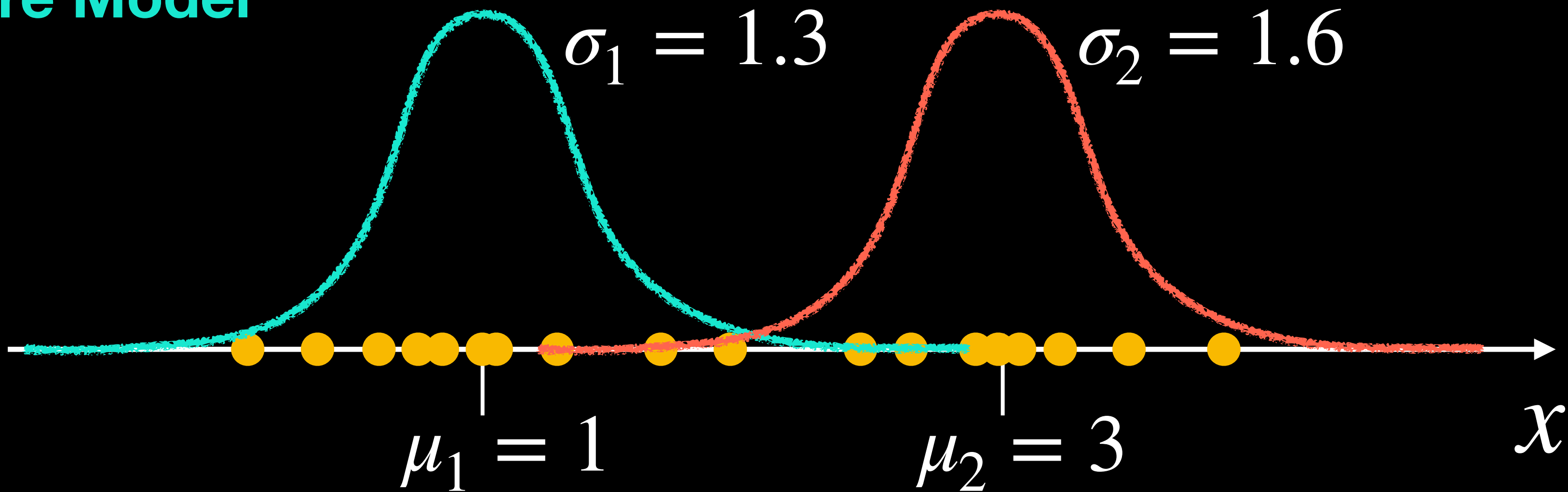
$$\sum_{k=1}^k \phi_k = 1$$

$$P(x^{(i)} | z^{(i)} = j) = \mathcal{N}(\mu_j, \sigma_j^2)$$

Gaussian Mixture Model

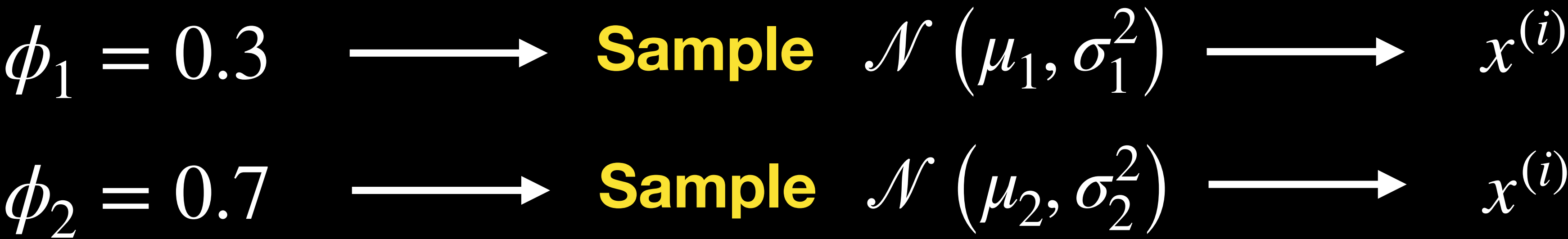
Dataset

x
1.2
3.2
4.3
3.2
...

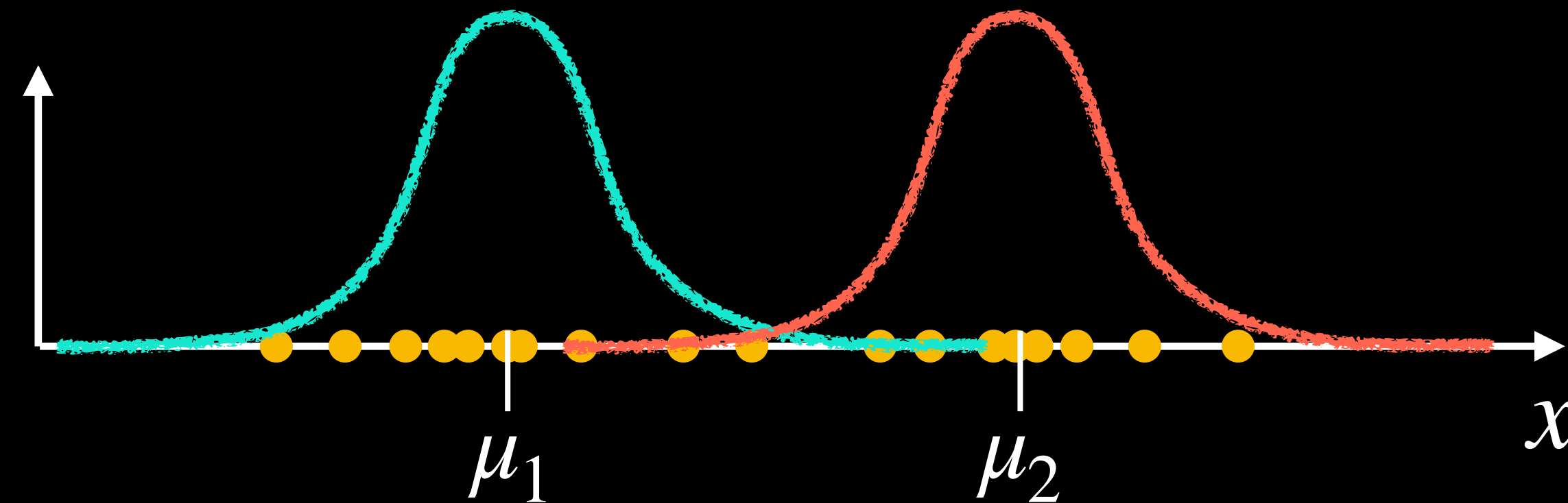


Intuition: if one were to ‘create’ the data

Sample



Expectation-Maximization (EM) Algorithm



E-Step: Guess $z^{(i)}$ given ϕ, μ, σ . Compute $P(z^{(i)} | x^{(i)}; \phi, \mu, \sigma)$

M-Step: Estimate ϕ, μ, σ using Maximum Likelihood Estimation

Expectation Step

E-Step: Guess $z^{(i)}$ given ϕ, μ, σ

$$\begin{aligned}w_j^{(i)} &\equiv P\left(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \sigma\right) = \frac{P\left(z^{(i)} = j, x^{(i)}\right)}{P\left(x^{(i)}\right)} \\&= \frac{P\left(x^{(i)} \mid z^{(i)} = j\right) P\left(z^{(i)} = j\right)}{P\left(x^{(i)}\right)} \\&= \frac{P\left(x^{(i)} \mid z^{(i)} = j\right) P\left(z^{(i)} = j\right)}{\sum_{s=1}^k P\left(x^{(i)} \mid z^{(i)} = s\right) P\left(z^{(i)} = s\right)}\end{aligned}$$

Expectation Step

E-Step: Guess $z^{(i)}$ given ϕ, μ, σ

$$w_j^{(i)} = P(z^{(i)} = j | x^{(i)}; \phi, \mu, \sigma)$$

$$= \frac{\mathcal{N}(\mu_j, \sigma_j^2) \underbrace{P(x^{(i)} | z^{(i)} = j)}_{\mathcal{N}(\mu_j, \sigma_j^2)} \underbrace{P(z^{(i)} = j)}_{\phi_j}}{\sum_{s=1}^k \underbrace{P(x^{(i)} | z^{(i)} = s)}_{\mathcal{N}(\mu_s, \sigma_s^2)} \underbrace{P(z^{(i)} = s)}_{\phi_s}}$$

Maximization Step

M-Step: Given $P(z^{(i)} = j) \equiv w_j^{(i)}$ estimate ϕ, μ, σ

Maximum Likelihood Estimation

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^n \log P(x^{(i)}; \phi, \mu, \sigma)$$

$$= \sum_{i=1}^n \log \sum_{z^{(i)}=1}^k P(x^{(i)} | z^{(i)}; \mu, \sigma) P(z^{(i)}; \phi) .$$

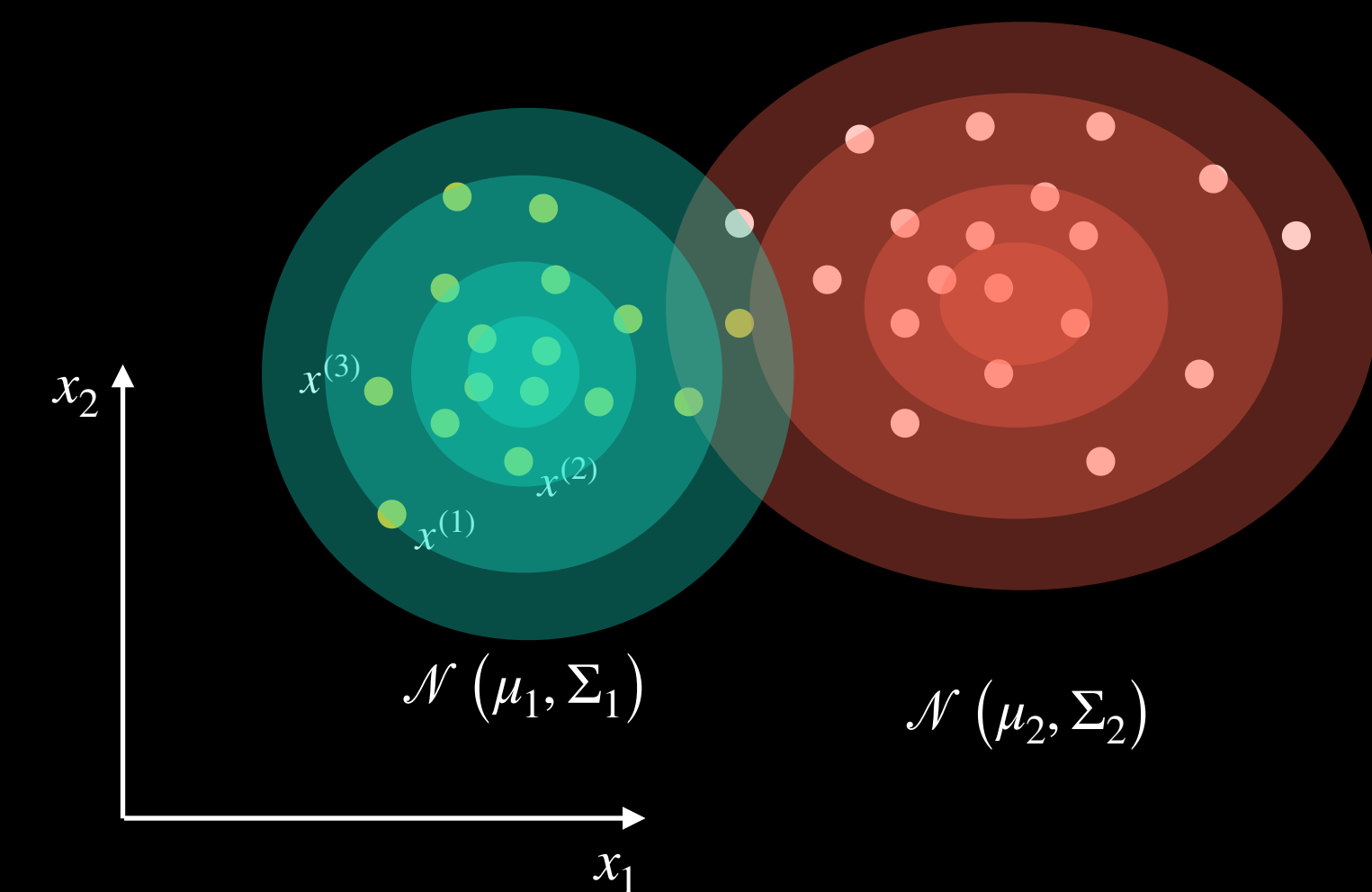
$$= \sum_{i=1}^n \log p(x^{(i)} | z^{(i)}; \mu, \sigma) + \log p(z^{(i)}; \phi)$$

if $z^{(i)}$ is known

Maximization Step

M-Step: Given $P(z^{(i)} = j) \equiv w_j^{(i)}$ estimate ϕ, μ, σ

If $z^{(i)}$ were known $\rightarrow$ **Gaussian Discriminant Analysis**



$$\phi_j = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{z^{(i)} = j\}$$

$$\mu_j = \frac{\sum_{i=1}^n \mathbf{1}\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^n \mathbf{1}\{z^{(i)} = j\}}$$

$$\Sigma_j = \frac{\sum_{i=1}^n \mathbf{1}\{z^{(i)} = j\} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n \mathbf{1}\{z^{(i)} = j\}}$$

Maximization Step

M-Step: Given $P(z^{(i)} = j) \equiv w_j^{(i)}$ estimate ϕ, μ, σ

Since $z^{(i)}$ is not known, we use **soft assignments** instead:

$$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)}$$

$$\mu_j = \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}}$$

$$\Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

EM Algorithm Summary

E-Step: Estimate $w_j^{(i)}$

$$w_j^{(i)} = \frac{\frac{\mathcal{N}(\mu_j, \Sigma_j)}{\phi_j} P(x^{(i)} | z^{(i)} = j) P(z^{(i)} = j)}{\sum_{s=1}^k \frac{\mathcal{N}(\mu_s, \Sigma_s)}{\phi_s} P(x^{(i)} | z^{(i)} = s) P(z^{(i)} = s)}$$

Iterate

M-Step: Estimate ϕ, μ, Σ

$$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)}$$

$$\mu_j = \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}}$$

$$\Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

Generate Synthetic Data

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# Seed for reproducibility
np.random.seed(42)

# Generate synthetic data from 3 Gaussian distributions
mean1, cov1 = [5, 5], [[1, 0], [0, 1]]
mean2, cov2 = [0, 0], [[1, 0], [0, 1]]
mean3, cov3 = [8, 1], [[1, 0], [0, 1]]

data1 = np.random.multivariate_normal(mean1, cov1, 100)
data2 = np.random.multivariate_normal(mean2, cov2, 100)
data3 = np.random.multivariate_normal(mean3, cov3, 100)

# Combine all data
data = np.vstack((data1, data2, data3))

# Number of clusters
k = 3
n, d = data.shape
```

Initialize Parameters

```
# Combine all data
data = np.vstack((data1, data2, data3))

# Number of clusters
k = 3
n, d = data.shape

# Initialize parameters
phi = np.ones(k) / k # Initial equal weights (mixing coefficients) for each cluster
mu = np.random.rand(k, d) * 10 # Random means
Sigma = np.array([np.eye(d) for _ in range(k)]) # Identity covariance matrices

# EM algorithm parameters
max_iter = 20
tol = 1e-6
log_likelihoods = []

# Visualization grid
x, y = np.meshgrid(np.linspace(-5, 15, 100), np.linspace(-5, 15, 100))
pos = np.dstack((x, y))
```

Expectation-Minimization Updates

$$w_j^{(i)} = \frac{\frac{\mathcal{N}(x^{(i)} | \mu_j, \Sigma_j)}{\sum_{s=1}^k \mathcal{N}(x^{(i)} | \mu_s, \Sigma_s)} \frac{\phi_j}{\sum_{s=1}^k \phi_s}}{\frac{\mathcal{N}(x^{(i)} | \mu_j, \Sigma_j)}{\sum_{s=1}^k \mathcal{N}(x^{(i)} | \mu_s, \Sigma_s)} \frac{\phi_j}{\sum_{s=1}^k \phi_s}}$$

$$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)}$$
$$\mu_j = \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}}$$
$$\Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

```
for iteration in range(max_iter):
    # E-step: compute responsibilities (w_j^(i))
    w = np.zeros((n, k))
    for j in range(k):
        w[:, j] = phi[j] * multivariate_normal.pdf(data, mean=mu[j], cov=Sigma[j])
    w /= w.sum(axis=1, keepdims=True)

    # M-step: update phi (mixing coefficients), mu (means), and Sigma (covariances)
    Nk = w.sum(axis=0)
    phi = Nk / n
    mu = np.dot(w.T, data) / Nk[:, np.newaxis]
    Sigma = np.zeros((k, d, d))

    for j in range(k):
        diff = data - mu[j]
        Sigma[j] = np.dot(w[:, j] * diff.T, diff) / Nk[j]

    # Log-likelihood calculation
    log_likelihood = np.sum(np.log(np.dot(w, phi)))
    log_likelihoods.append(log_likelihood)

    # Visualization of the current iteration
    plot_gmm(data, mu, Sigma, iteration + 1)

    # Check for convergence
    if iteration > 0 and abs(log_likelihood - log_likelihoods[-2]) < tol:
        print(f'Converged at iteration {iteration}')
        break
```